ARENAFIGHT ENTERPRISE TECHNICAL ARCHITECTURE REPORT

Comprehensive Architectural Specification, Physical Evidence Verification Engine & MCP Integration

Version: 2.5.0 Enterprise CLI | **Architecture:** Pure C++20 Native Standard | **Target Platforms:** Windows (Win32/x64) & Linux (POSIX x86_64/aarch64)

Author: Lunarmist-byte | **GitHub:** https://github.com/Lunarmist-byte | **LinkedIn:** https://www.linkedin.com/in/amal-s-kumar-ba69a1290/

1. Executive Summary & Design Philosophy

Modern generative AI software development faces a pervasive, fundamental flaw: **the hallucination crisis**. Conventional AI agent frameworks rely upon the Large Language Model to self-evaluate its own output. When an LLM asserts *'The unit tests pass'* or *'The firmware has been corrected'*, standard frameworks mark the objective complete without running a single physical compiler check, test runner, or process execution. This results in silent regressions, uncompilable source trees, and fabricated verification claims.

ArenaFight is an enterprise-grade autonomous execution and verification runtime engineered in modern C++20. It eliminates hallucinated completion through the non-negotiable **Level 1 Physical Dominance Rule**: model output is strictly classified as unverified hypothesis until substantiated by concrete operating system proof (process return codes, stderr logs, file modification timestamps, AST Quality of Service diagnostics, and SHA-256 cryptographic hashes).

ArenaFight operates with dual execution paradigms: functioning either as a self-contained, zero-dependency **Standalone Native CLI** (`arena`) with direct local hardware profiling and background service orchestration, or as a synchronized **OpenCode Connected Engine** supporting bi-directional MCP server tools (`arena mcp`) and native slash commands.

System Architecture Comparison

Architectural Dimension	Traditional AI Coding Agents	ArenaFight C++20 Runtime (v2.5.0)
Ground Truth Model	Model self-declaration ('I wrote the code and it compiles')	Level 1 Physical Evidence: OS exit codes, compiler logs, file presence
Verification Rigor	Unverified or single-pass loose regex checks	6 Physical Evidence Gates + Independent Adversarial Reviewer
Host Hardware Awareness	Hardcoded token limits, timeouts, and parallel workers	Real-time host assessment (RAM, CPU cores, CUDA VRAM) & dynamic limits
Acceleration Support	Opaque cloud APIs; fails or throttles host on local iGPU/CPU	NVIDIA CUDA auto-offload (`num_gpu: 99`) + CPU slow-variant throttles
Daemon Lifecycle	Fails with connection error if Ollama/server is offline	Autonomous path discovery & background daemon launch (no user prompts)
OpenCode Integration	None; disjointed independent CLI tools	Bi-directional MCP server (9 tools) + custom slash commands (/arena, /verify)
Security & Auditability	Arbitrary command execution; zero audit logs or path jails	Path Jail boundary, DLP secret redactor, FIPS 180-4 SHA-256 hash chain

2. Anti-Confabulation Invariants & Evidence Formalism

To enforce mathematical determinism across autonomous loops, ArenaFight formalizes evidence into an explicit mathematical lattice where physical evidence strictly dominates model inference.

Level	Evidence Class	Weight	Physical Sensor / Data Source	Dominance Invariant
Level 1	DIRECT_EXECUTION	1.00	OS process termination code 0, g++ stdout, ctest	Strictly dominates all other levels. Overrides 100% of LLM claims.
Level 2	ARTIFACT_VERIFICATION	0.85	Filesystem stat (size > 0), SHA-256 checksum match	Dominates Level 3-6. Proves physical creation on disk.
Level 3	TOOL_OUTPUT	0.70	Output streams from file_read, git diff, grep	Dominates Level 4-6. Empirical data from tool execution.
Level 4	PROJECT_METADATA	0.50	CMakeLists.txt, compile_commands.json, opencode.json	Dominates Level 5-6. Static project structure constraints.
Level 5	USER_INPUT	0.35	Explicit user prompt, CLI arguments, confirmation input	Establishes project objective; overridden by physical laws.
Level 6	MODEL_DERIVED	0.10	LLM inference, natural language assertions, chain-of-thought	Lowest priority. Pure hypothesis. Never accepted as ground truth.

The 6 Physical Evidence Verification Gates

Before any task, milestone, or session is marked completed, the independent `Verifier` evaluates all 6 gates:

Gate	Verification Gate Name	Physical Sensor & Inspection Protocol	Pass Criteria & Latch Behavior
Gate 1	Artifact Physical Presence	Target filesystem path check via std::filesystem::exists and file_size	File exists on disk, size > 0 bytes, permissions valid.
Gate 2	Clean Toolchain Compilation	Process execution of ninja, cmake --build, g++, or clang++	Compiler process exits with exit code 0; zero syntax or linker errors.
Gate 3	Automated Test Suite Pass	Process execution of ctest, test_suite.exe, or pytest	Test runner process exits with code 0; 100% assertions green.
Gate 4	Requirements Satisfaction	Dependency graph topological validation against session task tree	All dependent subtasks resolved without blocked or skipped states.
Gate 5	Security Sandbox & DLP	SecuritySandbox AST scanner, path jail validator, DLP regex engine	Zero path traversal escapes, zero unmasked secrets, clean risk score.
Gate 6	SHA-256 Audit & Contradictions	FIPS 180-4 SHA-256 hash chain verification + ClaimRegistry check	Cryptographic chain intact; zero unresolved contradictory claims.

3. Dual Processing Levels & Autonomous Hardware Assessment

ArenaFight dynamically bifurcates execution into two processing levels (`arena level [fast|perfect]`):

Operational Metric	FAST Processing Level (`--fast`)	PERFECT BOI Processing Level (`--perfect`)
Primary Objective	Rapid iteration, rapid linting, single-turn development	Exhaustive perfection, deep verification, self-healing remediation
Iteration Ceilings	Fixed baseline: 10 loops max	Dynamic: 25 to 60 loops depending on available RAM and CPU cores
Token Allocation	4,096 tokens max | 1,024 reasoning tokens	Dynamic: up to 16,384 - 32,768 tokens based on hardware headroom
Adversarial Review	Single pass (fast path checks)	Multi-pass cross-examination with synthetic bug injection
Self-Healing Remediation	Fast fail with actionable diagnostics	Autonomous task graph replanning & self-healing until 100% verified

SystemProfiler: Real-time Host PC Assessment & Acceleration Calibration

ArenaFight eliminates hardcoded limits by assessing host computing headroom via Win32 `GlobalMemoryStatusEx`, `GetNativeSystemInfo`, and Linux `/proc/meminfo`, `/proc/cpuinfo`. Limits are calculated on-the-fly:

- **CUDA Acceleration Active:** When an NVIDIA discrete GPU with driver support is discovered (e.g. RTX 4060 Laptop GPU), ArenaFight passes `num_gpu: 99` to model backends, caching all layers in high-bandwidth VRAM for 40-100+ tokens/sec throughput.
- **Integrated GPU / CPU Slow-Variant Protection:** When running on iGPUs (AMD Radeon Graphics, Intel Iris Xe) or CPU-only: ArenaFight displays a prominent terminal performance notice, caps tokens at 2048, throttles worker threads to prevent bus thrashing, and sets `num_gpu: 0` to safeguard against host lockup.

Hardware Inference Profile Matrix

Hardware Configuration	Inference Throughput	Memory Bus Latency	Adaptive Limit Policy
NVIDIA Dedicated GPU (CUDA)	40 to 110+ tokens/sec	< 25 ms first-token	Maximum parallel concurrency (4-8 workers), deep thinking enabled
Integrated Graphics (iGPU)	8 to 18 tokens/sec	120 to 250 ms first-token	Moderate concurrency (2 workers), token ceiling 2048 tokens
Host CPU Only (AVX2/AVX-512)	2 to 12 tokens/sec	300 to 900 ms first-token	Single worker thread (1 worker), extended 180s verification timeout

4. Native Model Context Protocol (MCP) Server Architecture

ArenaFight features a production-grade, native Model Context Protocol (MCP) server running over standard input/output (`arena mcp` / `arena --mcp`). It implements JSON-RPC 2.0 framing supporting both newline-delimited JSON and HTTP-style `Content-Length:` stream headers. This allows OpenCode and other MCP clients to directly invoke ArenaFight tools.

Tool Name	Input Schema & Parameters	Execution Function & Verification Protocol
arena_verify	`strict: boolean`	Executes all 6 Physical Evidence Verification Gates on workspace. Verifies build, test suites, file diffs, QoS, and security.
arena_goal	`task: string (required)`	Runs full autonomous goal loop with ability-based task assignment, tool execution, and self-healing replanning until all gates pass.
arena_plan	`task: string (required)`	Generates structured execution plan artifact with milestones, dependencies, assigned models, and independent verification criteria.
arena_profile	`force_cpu: boolean`	Inspects physical host hardware (RAM, CPU core topology, NVIDIA CUDA GPU) and computes dynamic non-hardcoded resource limits.
arena_benchmark	`models: array<string>`	Runs standardized 4-axis benchmark suite evaluating models across reasoning, coding invariants, verification, and throughput.
arena_qos	`path: string`	Static AST firmware audit: enforces zero-heap dynamic allocation in ISRs/loops, non-blocking timing, and hardware watchdog compliance.
arena_calculate	`calculation: string, args: object`	Solves electronic circuit parameters: Ohm's law (V/R/I/P), LED current limiter, resistor divider, I2C pullup, battery lifetime.
arena_security	`action: 'scan' | 'audit'`	Inspects workspace defense-in-depth posture, scans for DLP secret leaks, and verifies FIPS 180-4 SHA-256 cryptographic audit chain.
arena_think	`task: string, candidateA, candidateB`	Fast thinking-phase tournament: Disallows code generation and pits pure chain-of-thought architectural reasoning trees against each other.
arena_generate_image	`prompt, output_path, engine, width, height`	Generates visual assets via Fooocus SDXL, ComfyUI, or neural diffusion with physical magic byte header validation.
arena_generate_video	`prompt, output_path, engine, mode, frames, fps`	Generates video sequences via Wan AI 2.1 or ComfyUI with container box and duration verification.
arena_media_inspect	`path: string (required)`	Inspects physical evidence of media files: header magic bytes (PNG, JPEG, MP4), dimensions, container info, and SHA-256 hash.

5. Bi-Directional OpenCode Integration Engine

ArenaFight bridges seamlessly to the [OpenCode](https://opencode.ai) CLI ecosystem (`opencode`), providing a complete bi-directional integration model:

A. Bi-Directional Execution Modes

- **ArenaFight Calling OpenCode:** ArenaFight dispatches complex coding goals through OpenCode (`arena --opencode ""` or `arena opencode run ""`), discovering all installed models (`opencode-default`, `opencode/claude-3-7-sonnet`, `opencode/gemini-2.5-pro`, `opencode/deepseek-r1`, `opencode/qwen-2.5-coder`).
- **OpenCode Calling ArenaFight:** OpenCode agents and human developers invoke ArenaFight directly via native Model Context Protocol (MCP) tools, custom slash commands, or shell commands.

OpenCode Custom Slash Commands

Slash Command	Underlying Invocation	Operational Description & Verification Action
`/arena <task>`	`arena $ARGUMENTS`	Executes any ArenaFight goal, hardware calculation, plan, or subcommand directly from OpenCode terminal.
`/verify`	`arena verify`	Runs all 6 Physical Evidence Verification Gates across compiler, tests, diffs, and security. Enforces Level 1 dominance.
`/profile`	`arena profile`	Assesses physical host machine resources (CPU, RAM, CUDA GPU) and displays active dynamic resource limits.
`/benchmark`	`arena benchmark`	Runs the universal multi-model benchmark suite evaluating reasoning, coding, verification, and tokens/sec throughput.
`/qos [path]`	`arena qos [path]`	Audits embedded C++ firmware for zero-heap ISR compliance, non-blocking loops, and hardware watchdog timers.

B. Automated Workspace Synchronization (`arena opencode sync`)

Running `arena opencode sync` automatically writes all necessary configuration to bridge both runtimes:
1. **`opencode.json` & `.opencode/opencode.json`:** Registers `arena` as a local MCP server (`command: ["arena", "mcp"]`).
2. **`.opencode/commands/*.md`:** Generates Markdown command definitions for `/arena`, `/verify`, `/profile`, `/benchmark`, `/qos`.
3. **`.opencode/INSTRUCTIONS.md`:** Mandates that OpenCode agents run `arena_verify` before concluding any coding task.

6. Embedded Systems Studio & Hardware Intelligence

ArenaFight incorporates a dedicated microcontroller hardware registry and circuit engineering studio (`arena boards`, `arena scaffold`):

Microcontroller Board	Architecture & Core	Logic Level	Pinout Constraints & Hazard Rules
ESP32 (WROOM-32)	Xtensa Dual-Core 240MHz	3.3V Logic	Strapping pins: GPIO 0, 2, 12, 15. Input-only pins: GPIO 34, 35, 36, 39 (no pullups).
ESP32-S3 (DevKitC-1)	Xtensa LX7 Dual-Core + Vector	3.3V Logic	Native USB JTAG on GPIO 19/20. Strapping pins: GPIO 0, 45, 46. Octal PSRAM conflict pins.
Raspberry Pi Pico (RP2040)	ARM Cortex-M0+ Dual-Core	3.3V Logic	ADC pins restricted to GP26, GP27, GP28. Dual I2C/SPI flexible pin multiplexing.
STM32F401 (BlackPill)	ARM Cortex-M4F 84MHz (FPU)	3.3V (5V Tolerant)	Boot0 pin selection latch. NRST line decoupling requirements. Hardware floating-point.
Arduino Uno R3	Microchip ATmega328P 16MHz	5.0V Logic	5V logic hazard when interfacing with 3.3V sensors. Requires voltage divider calculation.

Static Firmware Quality of Service (QoS) & Circuit Calculations

- **Zero-Heap Dynamic Allocation Auditing:** Scans source code for `malloc`, `free`, `new`, `delete`, and `std::vector` inside Interrupt Service Routines (ISRs) and critical timing loops to prevent heap fragmentation panics.
- **Non-Blocking Loop Enforcer:** Flags `delay()` calls in cooperative multitasking environments, demanding `millis()`-based state machines.
- **Circuit Calculator (`arena calculate`):** Calculates Ohm's law, LED current-limiting resistors with E12/E24 series snapping, two-resistor voltage dividers for 5V-to-3.3V level shifting, I2C bus pullups, and battery deep-sleep life budgets.

7. Defense-in-Depth Security Sandbox & Cryptographic Audit

ArenaFight enforces multi-layer security protections to isolate autonomous tool execution:

Security Defense Layer	Threat Vector Mitigated	Deterministic Mitigation Engine
Path Jail Confinement	Directory traversal escapes (`../`, `../../etc/passwd`, Windows registry)	Canonicalizes all filesystem paths against workspace root. Rejects any reference traversing outside project boundary.
Command Risk Gatekeeper	Destructive shell execution (`rm -rf /`, `mkfs`, fork bombs, reverse shells)	4-tier command risk parser: SAFE (read-only), AUDITED (tracked), RESTRICTED (user confirmation required), FORBIDDEN (blocked).
Data Loss Prevention (DLP)	Accidental credential leaks in logs, prompts, or commit messages	Autonomous regex scanner masking API keys (OpenAI, Anthropic, AWS, GitHub), JWT bearer tokens, and private SSH keys.
Cryptographic Hash Chain	Unauthorized artifact tampering, covert code injection, log manipulation	Append-only `.arena/security_audit.jsonl` where each event contains SHA-256 hash of previous record (Merkle chain).

Autonomous Background Service Daemon (`ServiceDaemon`)

Instead of failing or demanding manual user intervention when local model servers are offline, ArenaFight features an autonomous service manager: autonomously searches standard installation paths on Windows and POSIX systems, spawns the daemon detached with hidden window, applies cooldown latches to avoid port conflicts, and probes health endpoints until online.

8. Complete Categorized Command & Slash Command Reference

ArenaFight v2.5.0 features a completely restructured, de-duplicated command reference across CLI and interactive REPL:

Category & Command	Inline Alias	Functional Description & Usage Syntax
1. Autonomous Execution	-	Autonomous self-healing goal pursuit (/goal <task>)
/goal	-	Milestone task graph generator (/plan <task>)
/plan	-	Interactive requirements interview (/grill-me <topic>)
/grill-me	**/duel**	Thinking tournament: Pure reasoning without code gen
/think	-	Run all 6 physical evidence verification gates on workspace
/verify		
2. Hardware & Processing	**/processing**	Set processing level: Fast (standard) vs Perfect Boi (adaptive)
/level	-	Assess physical host PC hardware & dynamic limits (/profile [cpu|cuda])
/profile	**/daemon**	Autonomous service daemon manager: auto-detect and start Ollama
/services		
3. AI Models & OpenCode	**/model**	List discovered models or set active model (/models set <name>)
/models	-	Display multi-dimensional capability breakdown matrix
/abilities	**/bench**	Universal benchmark across reasoning, code, verification & throughput
/benchmark	-	Switch mode (standalone native vs OpenCode connected)
/mode	-	Connect and synchronize with OpenCode bridge
/connect	-	OpenCode model discovery, execution and workspace sync
/opencode	-	Unsloth AI on-the-fly fine-tuning engine
/unsloth		
4. Security & Telemetry	-	View runtime execution mode, session state & verification stats
/status	**/telemetry**	Display terminal live telemetry observatory & gate pass rates
/metrics	-	Inspect defense-in-depth posture, scan workspace, set risk level
/security	-	Inspect tamper-evident SHA-256 audit log and verify hash chain
/audit		
5. Embedded Studio	**/board**	List or inspect microcontroller boards, pinouts, strapping pins
/boards	-	List supported sensors, actuators, and communication ICs
/components	-	Scaffold embedded project with Wokwi diagram and QoS firmware
/scaffold	-	Evaluate firmware for zero-heap, non-blocking QoS compliance
/qos	-	Calculate circuit values: Ohm's law, LED, divider, I2C pullup, battery
/calculate		
6. Context & Utilities	**/file**	Inspect, view, read, and attach workspace files to prompt (@file)
/files	-	Persist Knowledge Items (KI) or conventions into repo memory
/learn	-	Schedule periodic or delayed autonomous task execution
/schedule	-	Inspect and preview generated project artifacts
/artifacts	-	Manage agent skills and system/workspace operational rules
/skills, /rules	**/temperature**,	Adjust temperature (0.0-2.0) and max generation token limits
/temp, /tokens	**/max-tokens**	Interactive guided help, screen clear, and REPL exit
/guide, /clear, /exit	**/help-session**, **/quit**	

Interactive Context Mentions

Mention	Syntax	Context Expansion & Injection Behavior
File Injection	`@file <path>`	Reads target file from disk, attaches it to active session, injects contents directly into prompt context.
Working Tree Diff	`@git`	Executes git status and git diff; injects unstaged and staged repository changes into prompt.
Verified Claims	`@claims`	Injects ClaimRegistry evidence report and physically verified facts discovered during the session.
Session Memory	`@memory`	Injects historical session discoveries, user preferences, and persistent repository Knowledge Items.

9. Automated Verification Test Suite Scorecard (37/37 Passing)

ArenaFight undergoes rigorous regression testing across all architectural components. All 37 automated unit tests pass 100% green:

Test #	Automated Test Suite Specification	Targeted Subsystem	Result
1	Config loading and environment overrides	Core Configuration	PASSED
2	File tools (write, read, edit, delete)	File System Tools	PASSED
3	Search tools (list_directory, search_files)	Search Tools	PASSED
4	Process execution and destructive safety check	Process Executor	PASSED
5	Provider detection (Ollama, OpenRouter, Claude, OpenCode)	Provider Bridge	PASSED
6	Session serialization & disk persistence	Session Manager	PASSED
7	Dynamic model routing and independent verifier	Model Router	PASSED
8	Multi-step execution loop, failure, recovery & verif	Autonomous Agent	PASSED
9	Evidence hierarchy & dominance (Level 1 > Level 6)	Claim Registry	PASSED
10	Separation of facts from hypotheses	Memory Manager	PASSED
11	Consensus is supporting evidence, not proof	Verifier Core	PASSED
12	Contradiction detection & active resolution	Claim Registry	PASSED
13	Fresh verification & evidence invalidation on edit	Verifier Core	PASSED
14	Blind review & adversarial vulnerability inspection	Independent Reviewer	PASSED
15	All 6 verification gates & uncertainty handling	Verifier Core	PASSED
16	Automatic fallback to offline mock	Agent Engine	PASSED
17	Hardware Board Registry & pinout constraints	Embedded Studio	PASSED
18	Circuit Calculator (LED, Divider, I2C, Battery)	Embedded Studio	PASSED
19	Hardware Validator & Hazard Detection	Embedded Studio	PASSED
20	Hardware Scaffolder (Wokwi, PlatformIO, QoS)	Embedded Studio	PASSED
21	OpenCode Provider & Workspace integration	OpenCode Bridge	PASSED
22	Generation temperature and max tokens customization	Configuration	PASSED
23	Ability-based model evaluation, scoring & planning	Model Router	PASSED
24	Unsloth AI on-the-fly fine-tuning & dataset engine	Fine-Tuning	PASSED
25	File operations, context attachments & fine-tune data	Context Manager	PASSED
26	Fast thinking-phase tournament & early stopping	Model Router	PASSED
27	Pure CLI Terminal Telemetry & Verification Observatory	Terminal UI	PASSED
28	Universal compatibility with any models	Model Router	PASSED
29	Universal model benchmarking engine	Benchmark Engine	PASSED
30	CLI Standalone Native & OpenCode Connected modes	Runtime Engine	PASSED
31	Security Sandbox, path jail, DLP & SHA-256 audit	Security Sandbox	PASSED
32	Host hardware assessment, PC profiling & limits	System Profiler	PASSED
33	Two processing levels (Fast vs Perfect Boi)	Agent Engine	PASSED
34	Autonomous service detection & background daemon	Service Daemon	PASSED
35	CUDA acceleration & slow variant (iGPU/CPU) handling	System Profiler	PASSED
36	Native MCP server (stdio JSON-RPC) & OpenCode sync	MCP Server	PASSED
37	Generalized Media Tools (Foooocus, Wan AI) & Skills	Multimodal Media	PASSED

10. Project Attributions & Licensing

ArenaFight is authored and engineered by **Lunarmist-byte**. Licensed under custom All Rights Received terms: free for personal non-commercial experimentation and research; commercial deployment requires written authorization.
Author: Lunarmist-byte | **GitHub:** https://github.com/Lunarmist-byte | **LinkedIn:** https://www.linkedin.com/in/amal-s-kumar-ba69a1290/